

Programming Assignment: Hidden Markov Models

Deadline

To remain eligible for grade A in LAB2, all components required for your target grade and the individual Assignment Learning Journal must be submitted no later than **28 September 2026 at 19:00**. A late submission of any required component results in a maximum LAB2 grade of B.

Individual and pair work

You are encouraged to develop the programs in pairs, but you may also work individually. Groups larger than two are not permitted.

Even when the programs are developed in a pair, each student must submit an individual Assignment Learning Journal. You will be assessed individually and must be able to explain the relevant theory, your design decisions, your contribution, and the submitted implementations. Partners may refer to the same implementations and experiments, but each journal must be written individually and reflect the student's own understanding.

What you submit and how it is assessed

This assignment has two required forms of submission:

1. the programs required for your target grade, submitted to Kattis; and
2. an individual Assignment Learning Journal, submitted as a PDF in Canvas.

Completion of the required tasks and the Kattis results determine the highest grade range for which the work is eligible. The journal provides evidence of your technical understanding and is used to determine the grade within that range. A TA reviews the journal and code and assigns the final assignment grade. Your journal and code may later be used to prepare questions for your individual oral examination. The oral examination validates your individual understanding and may cap the final LAB2 grade.

1 INTRODUCTION TO HIDDEN MARKOV MODELS

A Hidden Markov Model (HMM) represents a sequential process in which the state of the system is not observed directly. Instead, each hidden state generates an observation according to a probability distribution. HMMs are used in areas such as robotics, speech and language

processing, and computational biology, where the underlying process must be inferred from noisy or indirect measurements.

In this assignment, all state and observation variables are discrete. Let X_t denote the hidden state and O_t the observation at time t . The model makes two central conditional-independence assumptions:

$$\begin{aligned} P(X_t \mid X_{1:t-1}) &= P(X_t \mid X_{t-1}), \\ P(O_t \mid X_{1:t}, O_{1:t-1}) &= P(O_t \mid X_t). \end{aligned}$$

The first is the first-order Markov assumption: given the previous state, the current state is independent of earlier states. The second says that, given the current state, the current observation is independent of the other states and observations.

For N hidden states and K possible observation symbols, an HMM is specified by $\lambda = (\mathbf{A}, \mathbf{B}, \boldsymbol{\pi})$:

$$\begin{aligned} \pi_i &= P(X_1 = i), & i &\in \{1, \dots, N\}, \\ a_{ij} &= P(X_{t+1} = j \mid X_t = i), & i, j &\in \{1, \dots, N\}, \\ b_i(k) &= P(O_t = k \mid X_t = i), & i &\in \{1, \dots, N\}, \quad k \in \{1, \dots, K\}. \end{aligned}$$

Thus, $\boldsymbol{\pi}$ is a $1 \times N$ initial-state distribution, $\mathbf{A}$ is an $N \times N$ transition matrix, and $\mathbf{B}$ is an $N \times K$ observation matrix. Every row of these probability matrices, and the vector $\boldsymbol{\pi}$, sums to one.

The joint probability of a state sequence $x_{1:T}$ and observation sequence $o_{1:T}$ factorizes as

$$P(x_{1:T}, o_{1:T}) = \pi_{x_1} b_{x_1}(o_1) \prod_{t=2}^T a_{x_{t-1}, x_t} b_{x_t}(o_t). \quad (1.1)$$

The assignment contains four programming tasks based on HMMs:

1. predict a distribution over the next observation;
2. evaluate the likelihood of an observed sequence;
3. decode the most likely hidden-state sequence; and
4. learn the model parameters from observations.

The final part applies these methods to classify fish species from their movement sequences.

Questions marked Q1–Q6 are assignment questions. Answer all six in Section J2 of your individual Assignment Learning Journal. Your answers should be concise, use your own words, and connect the concepts to your implementation whenever requested.

1.1 SUBMISSION PAGES

All Kattis problems are collected on the course assignment page:

<https://kth.kattis.com/courses/DD2380/ht26/assignments/zgpkex/problems>

The individual problem pages are also linked in the relevant sections below.

2 GRADE LEVELS E AND D

2.1 OVERVIEW

For eligibility in the E–D range, obtain an accepted Kattis submission for each of HMM0, HMM1, HMM2, and HMM3. These tasks implement prediction, evaluation, decoding, and parameter learning. Requirements are cumulative: higher grade levels also require these four accepted submissions.

For HMM0–HMM3, you may not use numpy; implement the required operations yourself using built-in data structures and arithmetic. The Kattis problems may use arbitrary HMM dimensions, not only those of the coin example below. In particular, do not assume that the number of hidden states equals the number of observation symbols.

Meeting the implementation requirements makes the work eligible for E or D. The distinction between E and D is based on the evidence of understanding, decisions, testing, and learning provided in the journal.

2.2 A RUNNING EXAMPLE

Suppose that the hidden state records which of two coins is used. Coin c_1 produces heads with probability 0.9, whereas coin c_2 is fair. Initially, either coin is selected with probability 0.5, and at every transition either coin is again selected with probability 0.5. The hidden states are therefore $\{c_1, c_2\}$ and the observation symbols are $\{H, T\}$.

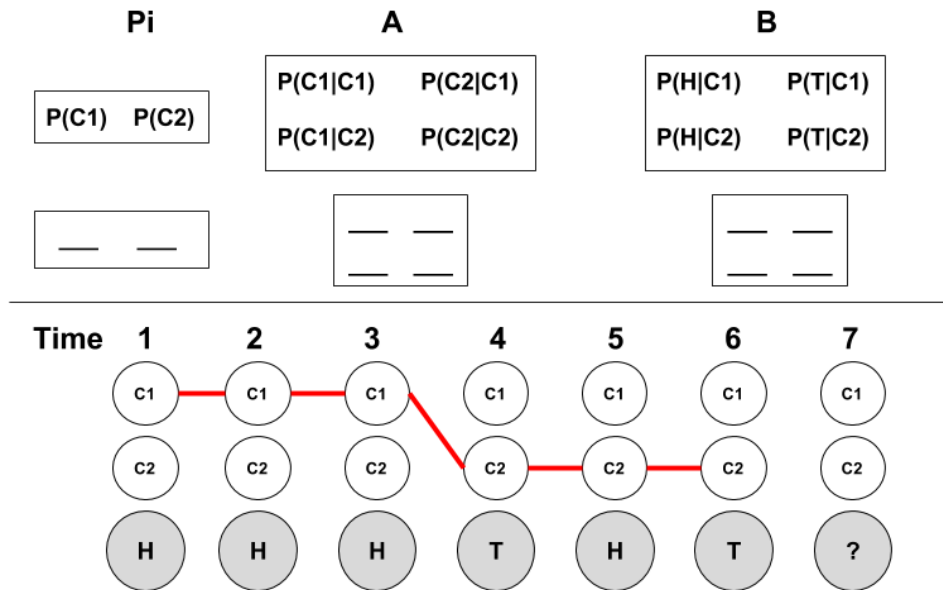


Figure 2.1: A two-state HMM in which the hidden state is the selected coin and the observation is heads or tails.

For example, marginalizing over the initially selected coin gives

$$P(O_1 = H) = \sum_{i=1}^2 P(O_1 = H \mid X_1 = c_i) P(X_1 = c_i) \quad (2.1)$$

$$= 0.9 \cdot 0.5 + 0.5 \cdot 0.5 = 0.7. \quad (2.2)$$

Q1. Specify π , $\mathbf{A}$, and $\mathbf{B}$ for the coin model, using the state order (c_1, c_2) and observation order (H, T) . Explain the meaning of one entry from each matrix or vector.

2.3 HMM0: NEXT-OBSERVATION DISTRIBUTION

Given a row vector $\mathbf{p}_t$ describing the current distribution over hidden states, first predict the next-state distribution and then the next-observation distribution:

$$\mathbf{p}_{t+1} = \mathbf{p}_t \mathbf{A}, \quad P(O_{t+1}) = \mathbf{p}_{t+1} \mathbf{B}. \quad (2.3)$$

For the Kattis problem, $\mathbf{p}_t$ is the supplied initial-state distribution π . Thus, the required next-observation distribution is $\pi \mathbf{A} \mathbf{B}$.

Implement these operations for <https://kth.kattis.com/courses/DD2380/ht26/assignments/zgpkex/problems/kth.ai.hmm0>.

Q2. For the coin model, use $\mathbf{p}_t = (0.8, 0.2)$ and calculate $\mathbf{p}_t \mathbf{A}$ and $\mathbf{p}_t \mathbf{A} \mathbf{B}$. Why does multiplication by $\mathbf{A}$ produce $(0.5, 0.5)$ regardless of the current state distribution? What property of $\mathbf{A}$ causes this, and what does each resulting vector represent?

2.4 HMM1: PROBABILITY OF AN OBSERVATION SEQUENCE

The forward algorithm evaluates the likelihood of an observation sequence $o_{1:T}$. Define

$$\alpha_t(i) = P(O_{1:t} = o_{1:t}, X_t = i \mid \lambda). \quad (2.4)$$

Initialize and update the forward values using

$$\alpha_1(i) = \pi_i b_i(o_1), \quad (2.5)$$

$$\alpha_t(i) = b_i(o_t) \sum_{j=1}^N \alpha_{t-1}(j) a_{ji}, \quad t = 2, \dots, T. \quad (2.6)$$

Finally,

$$P(O_{1:T} = o_{1:T} \mid \lambda) = \sum_{i=1}^N \alpha_T(i). \quad (2.7)$$

Implement the forward algorithm for <https://kth.kattis.com/courses/DD2380/ht26/assignments/zgpkex/problems/kth.ai.hmm1>.

Q3. In the forward recurrence, why may $P(O_t = o_t \mid X_t = i, O_{1:t-1} = o_{1:t-1})$ be replaced by $P(O_t = o_t \mid X_t = i)$? Why must the recurrence still sum over every possible predecessor state?

2.5 HMM2: MOST LIKELY SEQUENCE OF STATES

The Viterbi algorithm finds a single most likely hidden-state sequence. Let $\delta_t(i)$ be the probability of the most likely partial path that produces $o_{1:t}$ and ends in state i at time t . Then

$$\delta_1(i) = \pi_i b_i(o_1), \quad (2.8)$$

$$\delta_t(i) = b_i(o_t) \max_{j \in \{1, \dots, N\}} [\delta_{t-1}(j) a_{ji}]. \quad (2.9)$$

Store the maximizing predecessor for every t and i :

$$\psi_t(i) = \arg \max_{j \in \{1, \dots, N\}} [\delta_{t-1}(j) a_{ji}]. \quad (2.10)$$

After selecting $x_T^* = \arg \max_i \delta_T(i)$, recover the path by $x_t^* = \psi_{t+1}(x_{t+1}^*)$. Implement this algorithm for <https://kth.kattis.com/courses/DD2380/ht26/assignments/zgpkex/problems/kth.ai.hmm2>.

Q4. The forward and Viterbi recurrences have almost the same structure, but one uses a sum and the other a maximum. Explain how this changes both the meaning of the stored values and the final output. Why are the backpointers $\psi_t(i)$ needed?

2.6 HMM3: ESTIMATING MODEL PARAMETERS

Baum–Welch uses the forward–backward procedure to estimate $\lambda = (\mathbf{A}, \mathbf{B}, \boldsymbol{\pi})$ from an observation sequence. The backward quantity is

$$\beta_t(i) = P(O_{t+1:T} = o_{t+1:T} \mid X_t = i, \lambda), \quad (2.11)$$

with

$$\beta_T(i) = 1, \quad (2.12)$$

$$\beta_t(i) = \sum_{j=1}^N a_{ij} b_j(o_{t+1}) \beta_{t+1}(j). \quad (2.13)$$

The base case equals one because the probability of the empty sequence of future observations after time T is one.

For $t = 1, \dots, T - 1$, define the posterior probability of consecutive states i and j by

$$\gamma_t(i, j) = \frac{\alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}{P(O_{1:T} = o_{1:T} \mid \lambda)}, \quad (2.14)$$

and the posterior probability of state i by

$$\gamma_t(i) = \sum_{j=1}^N \gamma_t(i, j). \quad (2.15)$$

At the final time step,

$$\gamma_T(i) = \frac{\alpha_T(i)}{\sum_{j=1}^N \alpha_T(j)}. \quad (2.16)$$

The re-estimation equations are

$$\pi'_i = \gamma_1(i), \quad (2.17)$$

$$a'_{ij} = \frac{\sum_{t=1}^{T-1} \gamma_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}, \quad (2.18)$$

$$b'_i(k) = \frac{\sum_{t=1}^T \mathbf{1}(o_t = k) \gamma_t(i)}{\sum_{t=1}^T \gamma_t(i)}. \quad (2.19)$$

Repeat the forward–backward and re-estimation steps until the chosen convergence condition is met. Implement this procedure for <https://kth.kattis.com/courses/DD2380/ht26/assignments/zgpkex/problems/kth.ai.hmm3>. The Stamp tutorial [1] provides further implementation details. For Kattis, output only the re-estimated transition matrix $\mathbf{A}$ and observation matrix $\mathbf{B}$. Do not output the re-estimated initial-state distribution $\boldsymbol{\pi}$.

Q5. Explain why $\beta_T(i) = 1$. Then explain what $\gamma_t(i, j)$ and $\gamma_t(i)$ represent and why both are useful when re-estimating an HMM.

CONVERGENCE AND NUMERICAL STABILITY

Baum–Welch is an iterative optimization procedure. A practical stopping rule should use a measurable quantity, such as the improvement in log-likelihood, together with a maximum number of iterations. An overly loose convergence tolerance may stop training before the parameters are accurate enough for the Kattis tests. Verify the effect of your stopping rule using the supplied sample cases.

Directly multiplying many probabilities can underflow for long sequences. Scaling the forward and backward values at every time step, or performing the computation in log space where appropriate, avoids treating non-zero but very small probabilities as zero. If scaling is used, it must be applied consistently throughout the likelihood and re-estimation calculations.

Q6. Give a concrete convergence rule for your Baum–Welch implementation. Explain how you prevent numerical underflow and how your chosen method changes the computation of the sequence likelihood. Refer to the relevant functions in your code.

3 GRADE LEVEL C

For eligibility for C, first meet all E–D requirements and then investigate the behaviour of Baum–Welch empirically. The purpose is to connect the algorithm to convergence, initialization, model selection, and the amount of available data.

Canvas provides two observation sequences generated independently from the following HMM:

$$\mathbf{A} = \begin{pmatrix} 0.70 & 0.05 & 0.25 \\ 0.10 & 0.80 & 0.10 \\ 0.20 & 0.30 & 0.50 \end{pmatrix} \quad \mathbf{B} = \begin{pmatrix} 0.70 & 0.20 & 0.10 & 0.00 \\ 0.10 & 0.40 & 0.30 & 0.20 \\ 0.00 & 0.10 & 0.20 & 0.70 \end{pmatrix} \quad \boldsymbol{\pi} = (1 \ 0 \ 0)$$

The files are named `hmm_c_N1000.in` and `hmm_c_N10000.in`, according to their sequence lengths. They are separate realizations; neither sequence is derived from the other.

Required empirical investigation for C or above

Use your own Baum–Welch implementation to perform all three investigations below. Keep the observation alphabet at four symbols, as specified by the data. Record the experimental setup and use the same stopping rule when a comparison requires it. Record the exact initialization matrices used. If an initialization is generated randomly, also record the generation procedure and random seed.

- Sequence length and convergence.** Train a three-state HMM on both supplied sequences, initialized with

$$\mathbf{A}_0 = \begin{pmatrix} 0.54 & 0.26 & 0.20 \\ 0.19 & 0.53 & 0.28 \\ 0.22 & 0.18 & 0.60 \end{pmatrix}, \quad \mathbf{B}_0 = \begin{pmatrix} 0.50 & 0.20 & 0.11 & 0.19 \\ 0.22 & 0.28 & 0.23 & 0.27 \\ 0.19 & 0.21 & 0.15 & 0.45 \end{pmatrix}, \quad \boldsymbol{\pi}_0 = (0.3 \ 0.2 \ 0.5).$$

Compare the iterations required, final log-likelihood per observation, learned parameters, and sensitivity to sequence length.

2. **Initialization.** Compare at least three initializations, including a uniform model, a model containing exact zeros (for example, a diagonal transition matrix with $\pi = (0, 0, 1)$), and a non-symmetric model reasonably close to the generating parameters. Explain symmetry, structural zeros, local optima, or other behaviour you observe.
3. **Number of hidden states and evaluation.** Fit models with two, three, and four hidden states. Use the same initialization procedure and stopping rule for all three model sizes. If you use multiple random restarts, use the same number for each model size and the same criterion for selecting the retained model. Train each model using `hmm_c_N10000.in`. Without performing any further parameter updates, evaluate each learned model on `hmm_c_N1000.in`. Report both the training and evaluation log-likelihood per observation. Explain why raw training likelihood alone is not sufficient for selecting the number of hidden states. For the three-state model, also compare the learned and generating parameters while accounting for arbitrary permutations of the hidden-state labels.

Summarize the most important evidence in J3 of your journal. A compact table is welcome; extensive raw output is not. State enough detail for a TA to understand and, in principle, reproduce the comparisons.

4 GRADE LEVELS B AND A

4.1 FISHING DERBY: CLASSIFYING FISH

For eligibility in the B–A range, first meet the E–D and C requirements. You must then implement an agent that classifies fish species from sequences of observed movements.

Each scenario contains 70 fish drawn from 7 possible species. The fish are not necessarily distributed evenly across these species, and the distribution may differ between scenarios. Each scenario lasts for at most 180 time steps. The game may end earlier when all 70 fish have been guessed. At each time step while the game is running, your guess method receives one movement observation for every fish. The observations are integers representing movement in the directions UP, DOWN, LEFT, RIGHT, UP–LEFT, UP–RIGHT, DOWN–LEFT, and DOWN–RIGHT. The method may return no guess, or it may return one pair (`fish_id`, `fish_type`).

A particular species may exhibit a small number of recurring swimming patterns, and different species may exhibit different variations of these patterns. This should inform your choice of hidden states and model structure.

After the first guess for a fish, the `reveal` method receives its true type, allowing your agent to use the newly labelled sequence in later decisions. A correct first guess earns one point, whereas an incorrect guess earns zero points; incorrect guesses do not incur an additional penalty. Only the first guess for each fish affects the score. Do not guess the same fish more than once: a repeated guess does not produce an additional scored fish and wastes a limited time step. The Kattis score is the total number of correct guesses across the test scenarios.

Waiting before guessing provides longer observation sequences and may make individual classifications more reliable. However, guessing earlier provides labelled sequences that can improve the species models used for later fish. Since at most one fish may be guessed at each time step, the agent must also reserve enough remaining steps for the fish it intends to classify. If only 70 steps remain and no fish has yet been guessed, it must guess a new fish on every

remaining step to attempt all 70. Your strategy should therefore consider both when and in what order to guess.

Your solution must decide how to represent and train the species models, how to compare a fish's sequence with those models, and how to balance additional observations against the value of obtaining labelled examples earlier. The test scenarios differ from the local example. The program is restarted between scenarios, so models, labelled examples, and other learned information cannot be transferred from one scenario to the next.

4.2 PROVIDED CODE AND INTERFACE

Use the Python skeleton supplied in Canvas. Implement the methods of `PlayerControllerHMM` in `player.py`; you may create additional Python files and import them from `player.py`. Other skeleton files may be changed for local experimentation, but Kattis overwrites them. Avoid using standard input and standard output in your player code; use standard error for debugging.

To run the supplied local scenario from the skeleton directory, use

```
python3 main.py < sequences.json
```

The local graphical application runs with CPython, whereas Kattis uses PyPy. Consequently, relative performance may differ. The per-call time limit is 5 seconds. Although `numpy` is available for this part, operations on ordinary Python lists may run faster under PyPy. Measure the behaviour of your own implementation rather than assuming either representation is faster.

The complete Kattis description and submission page are available at [the Kattis Fishing Derby problem page](#).

4.3 GRADE REQUIREMENT

Kattis requirement: a Fishing Derby score of at least 250.

Meeting this threshold, together with the cumulative E–D and C requirements, makes the work eligible for B or A. The distinction between B and A is based on the quality of the student's analysis, evidence, comparison of alternatives, and understanding of limitations, as documented in the journal. Grade A also requires on-time submission.

Repeatedly changing constants or model dimensions until the required score is reached is not by itself sufficient evidence for B or A. You should be able to explain why your modelling and decision strategy should work, show evidence of its effect, and identify conditions under which it may fail.

5 ASSIGNMENT LEARNING JOURNAL

The Assignment Learning Journal is an individual, concise account of what you implemented, why you made important decisions, and what you learned. It is not a diary or a full technical report. Connect your explanations to your submitted code, experiments, and observations.

Completion of the required tasks and the Kattis results determine the highest eligible grade range. The journal provides evidence of your individual understanding within that range. A TA reviews the journal and code and assigns the final grade. The journal and code may also be used to prepare questions for the individual oral examination.

5.1 FORMAT AND SUBMISSION

Use the headings J0–J6 and submit the journal as instructed in Canvas. Most journals will be approximately 800–1200 words. The maximum is 1500 words, excluding the question prompts, J0 and J6. Students aiming for E or D will normally be closer to the lower end, while students

aiming for C, B, or A may need the upper end. Short paragraphs and bullet points are welcome. Do not paste source code or screenshots; refer to files, classes, or functions by name. One small results table is permitted when it makes an empirical comparison clearer.

If you worked with a partner, the implementations and experiments may be shared, but the journal must be written individually. Shared facts may overlap; your explanations, contribution, and learning must be your own. Assessment concerns technical understanding, not writing style.

J0 – Submission record. Give your name and KTH email, your partner’s name (or “none”), and the grade you are aiming for. List the Kattis submission identifiers for HMM0, HMM1, HMM2, and HMM3. If aiming for B or A, also give the Fishing Derby submission identifier and score.

J1 – Solution overview (about 100–150 words). Explain how your implementations connect the four programming tasks in this assignment: prediction, sequence likelihood, state-sequence decoding, and parameter learning. Name the important files, classes, or functions and distinguish provided code from what you implemented or substantially changed. If aiming for B or A, also summarize how the classifier uses observations and revealed labels to produce its guesses.

J2 – Assignment questions. Answer Q1–Q6, normally in about 40–80 words each. Use a small calculation, formula, trace, or code reference when helpful. Explain in your own words rather than only repeating a definition.

J3 – Decisions and evidence. Describe two important decisions concerning, for example, representation, scaling, convergence, initialization, model dimensions, efficiency, testing, classification, or the timing of guesses. For each, give the goal, alternatives considered, choice and reason, code location, evidence, and a trade-off or limitation. Evidence may be a controlled comparison, local test, trace, measurement, Kattis result, or failure case.

If aiming for C or above, use this section to summarize the required empirical investigation and connect at least one of your decisions to its results. At least one decision must include a revised approach, unexpected behaviour, important boundary case, remaining limitation, or meaningful alternative you evaluated.

J4 – Advanced analysis (required when aiming for B or A). Build on one of the decisions involved in the Fishing Derby classifier. Analyze it in more depth: explain how it works, why it should help, evidence of its effect, and a limitation or trade-off. Describe a reasonable alternative. For grade A, critically compare the alternatives using concrete evidence and justify your choice under the assignment’s constraints.

J5 – Individual learning and contribution (about 100–150 words). State what you personally contributed. Describe the most important thing you understand now that you did not understand before, the hardest issue you resolved, and one limitation or open question that remains.

J6 – Use of AI tools. State whether and how you used generative AI or other automated assistance in this assignment, both for the code and this journal. Identify what it was used for and explain how you verified and understood any resulting suggestions or code. If you did not use such tools, state this explicitly. Use of these tools is permitted as support, but you remain responsible for all submitted code and text and must be able to explain them during the individual oral examination.

REFERENCES

- [1] M. Stamp, *A Revealing Introduction to Hidden Markov Models*, San José State University, 12 April 2021. <https://www.cs.sjsu.edu/faculty/stamp/RUA/HMM.pdf>
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, Prentice Hall.